

**K L UNIVERSITY**  
**FRESHMAN ENGINEERING DEPARTMENT**

**A Project Based Lab Report**

**On**

**SMART BANKING MANAGEMENT SYSTEM**

**SUBMITTED BY:**

| ID NO      | NAME              |
|------------|-------------------|
| 2500030011 | G.RAM CHAND       |
| 2500030042 | P.DINESH KUMAR    |
| 2500030005 | E.PRADIPTHA REDDY |
| 2500030020 | K.RISHI PREETHAM  |

**UNDER THE ESTEEMED GUIDANCE OF**

**Mr. REPUDI PITCHIAH**

**ASSISTANT PROFESSOR**



**KL UNIVERSITY**  
Green fields, Vaddeswaram – 522 502  
Guntur Dt., AP, India.

# DEPARTMENT OF BASIC ENGINEERING SCIENCES



## CERTIFICATE

This is to certify that the project based laboratory report entitled “**SMART BANKING MANAGEMENT SYSTEM**” submitted by **G.Ram Chand, P.Dinesh Kumar, E.Pradiptha Reddy, K.Rishi Preetham** bearing Regd. No. **2500030011, 2500030042, 2500030005, 2500030020** to the **Department of COMPUTER SCIENCE ENGINEERING , KL University** in partial fulfillment of the requirements for the completion of a project based Laboratory in “**Python Programming (25SC2007)**” course in I B Tech II Semester, is a Bonafede record of the work carried out by him/her under my supervision during the academic year 2025 – 2026.

PROJECT SUPERVISOR

Mr. REPUDI PITCHIAH

HEAD OF THE DEPARTMENT

Dr. D. HARITHA

# ACKNOWLEDGEMENT

It is great pleasure for me to express my gratitude to our honorable President.

**Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the project-based laboratory report.

I express sincere gratitude to our Director **Dr. A. Jagdeesh** for his administration towards our academic growth.

I express sincere gratitude to our Coordinator and HOD-BES **Dr. D. Haritha** for her leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank you for providing us with the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Mr. Repudi Pitchiah** for her novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, we are pleased to acknowledge our indebtedness to all those who devoted themselves directly or indirectly to making this project a success.

**Name: G.Ram Chand**

**P.Dinesh Kumar**

**E.Pradiptha Reddy**

**K.Rishi Preetham**

**ID No: 2500030011**

**2500030042**

**2500030005**

**2500030020**

# **ABSTRACT**

The Smart Banking Management System is a Python-based application designed to simplify and automate basic banking operations. The system enables users to perform essential activities such as account creation, deposit, withdrawal, balance inquiry, and transaction history management through a user-friendly, menu-driven interface.

This project is developed using object-oriented programming concepts in Python, where classes and data structures like dictionaries and lists are used to efficiently store and manage customer account information. The system ensures accuracy, reduces manual errors, and provides quick access to financial data.

Although it is a console-based application, it demonstrates the core functionalities of modern banking systems and serves as a foundation for advanced features such as database integration, user authentication, and graphical user interfaces. The project highlights how Python can be effectively used to build simple, secure, and scalable financial management applications.

## **INDEX**

| <b>S.NO</b> | <b>TITLE</b>                   | <b>PAGE NO</b> |
|-------------|--------------------------------|----------------|
| 1           | Introduction                   | 6              |
| 2           | Aim of the Project             | 7              |
| 2.1         | Advantages & Disadvantages     | 8              |
| 2.2         | Future Implementation          | 8              |
| 3           | System Requirements            | 9              |
| 4           | Data Flow Diagram              | 10             |
| 5           | Algorithm for each module      | 11             |
| 6           | Implementation                 | 12-19          |
| 7           | Integration and System Testing | 20-22          |
| 8           | Conclusion                     | 23             |

# 1.INTRODUCTION

In today's digital world, banking systems play a vital role in managing financial transactions efficiently and securely. Traditional banking methods involve manual processes that are time-consuming, error-prone, and less efficient. With the advancement of technology, there is a growing need for automated systems that can handle banking operations quickly and accurately.

The Smart Banking Management System is a Python-based application developed to simulate basic banking functionalities in a simple and effective manner. This system allows users to perform operations such as creating an account, depositing money, withdrawing funds, checking account balance, and viewing transaction history through a menu-driven interface.

The project is designed using object-oriented programming principles, making it modular, reusable, and easy to maintain. Data structures like dictionaries and lists are used to store and manage account details efficiently. The system focuses on providing user-friendly experience while ensuring accuracy and reliability in transactions.

This project serves as a foundational model for understanding how banking systems work and how programming can be used to automate financial services. It can be further enhanced by integrating databases, security features, and graphical user interfaces to make it more robust and practical for real-world applications.

## **2.AIM**

The aim of this project is to develop a Smart Banking Management System using Python that automates basic banking operations such as account creation, deposit, withdrawal, balance inquiry, and transaction tracking. The system is designed to provide a simple, efficient, and user-friendly platform for managing banking activities while ensuring accuracy and reducing manual effort.

**The modules in this project are –**

- Account Creation Module
- Deposit Module
- Withdrawal Module
- Balance Inquiry Module
- Transaction History Module
- Account Verification Module
- User Interface (Menu) Module

## **2.1.ADVANTAGES**

- The system is user-friendly and easy to operate, allowing users to perform banking operations like deposit, withdrawal, and balance checking quickly without requiring advanced technical knowledge.
- It reduces manual work and minimizes human errors by automating banking processes, thereby improving accuracy, efficiency, and saving time in managing financial transactions.
- The system maintains proper records of all transactions and account details, making it easier to track financial activities and ensuring better data management and organization.

## **2.1.DISADVANTAGES**

- The system is a basic console-based application and lacks advanced security features such as encryption and authentication, making it less suitable for real-world banking use.
- It does not use a database for permanent data storage, so all account information is lost once the program is closed, which limits its practical usability.
- The system has a simple interface and limited functionality, as it does not support online transactions, multi-user access, or integration with modern banking technologies.

## **2.2.FUTURE IMPLEMENTATION**

The Smart Banking Management System can be further enhanced by adding advanced features to make it more practical and suitable for real-world applications. A database such as MySQL or SQLite can be integrated to store customer and transaction data permanently, ensuring data is not lost after program execution. Security can be improved by implementing user authentication, password protection, and encryption techniques to safeguard sensitive information. Additionally, the system can be upgraded from a console-based application to a graphical user interface or web-based platform using technologies like Tkinter or Flask, which would provide a more interactive and user-friendly experience. Further improvements may include support for online transactions, multi-user access, and mobile banking integration.



## **3.SYSTEM REQUIREMENTS**

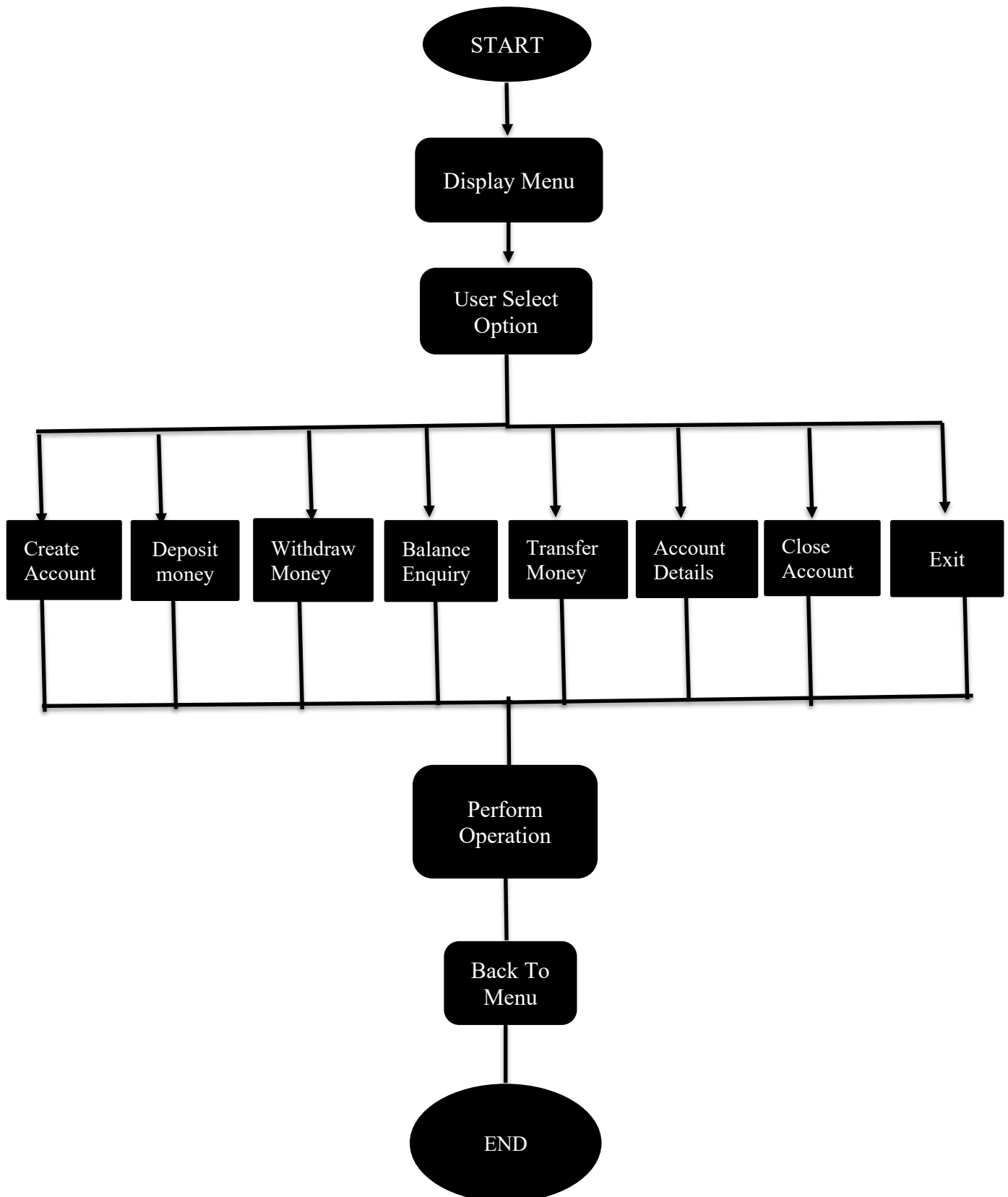
### **SOFTWARE REQUIREMENTS**

- Python 3.x (for developing and running the application)
- Any IDE or code editor such as VS Code, PyCharm, or IDLE
- Operating System: Windows, Linux, or macOS
- Basic Python libraries (no external libraries required)

### **HARDWARE REQUIREMENTS**

- Processor: Minimum Intel i3 or equivalent
- RAM: Minimum 4 GB
- Hard Disk: At least 500 MB free space
- Input Devices: Keyboard and Mouse
- Output Devices: Monitor

## 4.DATA FLOW DIAGRAM



## 5.ALGORITHM

1. Start the program
2. Display menu options
3. Accept user choice
4. If choice = Create Account
  - Get customer details
  - Save details into file
5. If choice = Deposit
  - Get account number
  - Add amount
  - Update file
6. If choice = Withdraw
  - Get account number
  - Check balance
  - Deduct amount if sufficient
7. If choice = Balance Enquiry
  - Display balance
8. If choice = Fund Transfer
  - Get sender and receiver account numbers
  - Check sender balance
  - Transfer amount
9. If choice = Close Account
  - Remove account record
10. Repeat until Exit
11. STOP



## 6.IMPLEMENTATION

```
import os

FILE_NAME =
"bankdata.txt"

# Function to read all
accounts
def read_accounts():
    accounts = []
    if
os.path.exists(FILE_
NAME):
    with
open(FILE_NAME,
"r") as file:
        for line in file:
            data =
line.strip().split(",")
            if len(data)
== 4:

accounts.append({

"accno": data[0],

"name": data[1],

"phone": data[2],

"balance":
float(data[3])
        })
    return accounts

# Function to write all
accounts
def
write_accounts(accou
nts):
    with
```

```

open(FILE_NAME,
"w") as file:
    for acc in
accounts:

file.write(f'{acc['accn
o']},{acc['name']},{ac
c['phone']},{acc['bala
nce']}\n")

# Create new account
def create_account():
    accounts =
read_accounts()
    accno =
input("Enter Account
Number: ")

    for acc in accounts:
        if acc["accno"]
== accno:
            print("Account
already exists!")
            return

    name =
input("Enter Name: ")
    phone =
input("Enter Phone
Number: ")
    balance =
float(input("Enter
Initial Deposit
Amount: "))

    accounts.append({
        "accno": accno,
        "name": name,
        "phone": phone,
        "balance":
balance
    })

write_accounts(accou
nts)

```

```

    print("Account
Created
Successfully!")

# Deposit money
def deposit_money():
    accounts =
read_accounts()
    accno =
input("Enter Account
Number: ")
    amount =
float(input("Enter
Deposit Amount: "))

    for acc in accounts:
        if acc["accno"]
== accno:
            acc["balance"]
+= amount

write_accounts(accou
nts)
    print("Deposit
Successful!")
    return

    print("Account Not
Found!")

# Withdraw money
def
withdraw_money():
    accounts =
read_accounts()
    accno =
input("Enter Account
Number: ")
    amount =
float(input("Enter
Withdrawal Amount:
"))

    for acc in accounts:
        if acc["accno"]
== accno:

```

```

        if
acc["balance"] >=
amount:

acc["balance"] -=
amount

write_accounts(accou
nts)

print("Withdrawal
Successful!")
    else:

print("Insufficient
Balance!")
    return

    print("Account Not
Found!")

# Balance enquiry
def
balance_enquiry():
    accounts =
read_accounts()
    accno =
input("Enter Account
Number: ")

    for acc in accounts:
        if acc["accno"]
== accno:
            print("Account
Balance:",
acc["balance"])
            return

    print("Account Not
Found!")

# Display account
details
def account_details():
    accounts =
read_accounts()

```



```

    accno =
input("Enter Account
Number: ")

    for acc in accounts:
        if acc["accno"]
== accno:
            print("\n---
Account Details ---")
            print("Account
Number:",
acc["accno"])
            print("Name:",
acc["name"])

print("Phone:",
acc["phone"])

print("Balance:",
acc["balance"])
        return

    print("Account Not
Found!")

# Transfer money
def transfer_money():
    accounts =
read_accounts()
    sender =
input("Enter Sender
Account Number: ")
    receiver =
input("Enter Receiver
Account Number: ")
    amount =
float(input("Enter
Amount to Transfer:
"))

    sender_acc = None
    receiver_acc =
None

    for acc in accounts:
        if acc["accno"]

```

```

== sender:
    sender_acc =
acc
    if acc["accno"]
== receiver:
    receiver_acc =
acc

    if sender_acc is
None or receiver_acc
is None:
        print("Sender or
Receiver Account Not
Found!")
        return

    if
sender_acc["balance"]
>= amount:

sender_acc["balance"]
-= amount

receiver_acc["balance
"] += amount

write_accounts(accou
nts)
    print("Money
Transferred
Successfully!")
    else:

print("Insufficient
Balance in Sender
Account!")

# Close account
def close_account():
    accounts =
read_accounts()
    accno =
input("Enter Account
Number to Close: ")

    for acc in accounts:

```

```

        if acc["accno"]
== accno:

accounts.remove(acc)

write_accounts(accounts)

        print("Account
Closed
Successfully!")
        return

        print("Account Not
Found!")

# Main Menu
def main():
    while True:
        print("\n=====
SMART BANKING
MANAGEMENT
SYSTEM =====")
        print("1. Create
Account")
        print("2. Deposit
Money")
        print("3.
Withdraw Money")
        print("4. Balance
Enquiry")
        print("5. Transfer
Money")
        print("6. Account
Details")
        print("7. Close
Account")
        print("8. Exit")

    print("=====
=====
=====
")

    choice =
input("Enter your
choice (1-8): ")

```

```

        if choice == "1":

create_account()
        elif choice ==
"2":

deposit_money()
        elif choice ==
"3":

withdraw_money()
        elif choice ==
"4":

balance_enquiry()
        elif choice ==
"5":

transfer_money()
        elif choice ==
"6":

account_details()
        elif choice ==
"7":

close_account()
        elif choice ==
"8":
            print("Thank
you for using Smart
Banking System!")
            break
        else:
            print("Invalid
Choice! Try again.")

# Run Program
main()

```

## 7.INTEGRATION AND SYSTEM TESTING

### OUTPUTS

#### Screen Shots:

```
===== SMART BANKING MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Balance Enquiry
5. Transfer Money
6. Account Details
7. Close Account
8. Exit
=====
Enter your choice (1-8): 1
Enter Account Number: 2500030011
Enter Name: Ram Chand
Enter Phone Number: 9014498051
Enter Initial Deposit Amount: 25000
Account Created Successfully!
```

```
===== SMART BANKING MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Balance Enquiry
5. Transfer Money
6. Account Details
7. Close Account
8. Exit
=====
Enter your choice (1-8): 2
Enter Account Number: 2500030011
Enter Deposit Amount: 2000
Deposit Successful!
```

```
===== SMART BANKING MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Balance Enquiry
5. Transfer Money
6. Account Details
7. Close Account
8. Exit
=====
Enter your choice (1-8): 3
Enter Account Number: 2500030011
Enter Withdrawal Amount: 2500
Withdrawal Successful!
```

```
===== SMART BANKING MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Balance Enquiry
5. Transfer Money
6. Account Details
7. Close Account
8. Exit
=====
Enter your choice (1-8): 4
Enter Account Number: 2500030011
Account Balance: 24500.0
```

```

===== SMART BANKING MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Balance Enquiry
5. Transfer Money
6. Account Details
7. Close Account
8. Exit
=====
Enter your choice (1-8): 5
Enter Sender Account Number: 2500030011
Enter Receiver Account Number: 2500080062
Enter Amount to Transfer: 5000
Money Transferred Successfully!

```

```

===== SMART BANKING MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Balance Enquiry
5. Transfer Money
6. Account Details
7. Close Account
8. Exit
=====
Enter your choice (1-8): 6
Enter Account Number: 2500080062

--- Account Details ---
Account Number: 2500080062
Name: Kruthika
Phone: 6303660717
Balance: 25000.0

```

```

===== SMART BANKING MANAGEMENT SYSTEM =====
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Balance Enquiry
5. Transfer Money
6. Account Details
7. Close Account
8. Exit
=====
Enter your choice (1-8): 7
Enter Account Number to Close: 2500030011
Account Closed Successfully!

```

## **8.CONCLUSION**

The Smart Banking Management System project successfully provides a digital solution for handling banking operations like account creation, deposits, withdrawals, balance enquiry, money transfer, and account closure. This project reduces paperwork, saves time, and improves accuracy in banking processes. It also demonstrates effective use of Python programming concepts such as functions, loops, conditional statements, and file handling. The project can be further enhanced by adding password authentication, GUI interface, and database connectivity.